

CSE235 Week 6: Lists and Tuples

Lists and tuples are used for storing collections of data in Python. Both are sequential containers, meaning that they preserve to order of their elements.

Assert

Throughout these examples, and future examples, `assert` statements will be used to demonstrate the state of values in the code. An assert statement will cause the program to halt execution should the condition fail.

```
assert 1 == 1
x = 3
# here, I'm communicating that x - 2 is 1
assert x - 2 == 1

# a condition that returns false will cause
# cause the program to stop executing immediately
assert 1 == 2

# assert can also be given a custom message to print
assert 1 == 2, "1 does not equal 2"
```

Lists

A list in Python is similar to an array in most programming languages, except that a Python list can contain a mix of data types. Arrays in languages like C++ and Java can only store elements of the same data type. Another difference is that Python lists do not have a fixed size like arrays in C++ and Java do. Elements can be added to lists at will. Below is a basic list:

```
# list is the name of the function to create a list
# therefore, lst is being used for the name of this
# generic list variable
```

```
lst = [0, "hello", True, "banana", 4.7]
```

As you can see, the list contains an integer, a string, a Boolean, another string, and a float. Another thing to note is that while a new line ends most statements in Python, a list can span multiple lines. Anything between the two square brackets is assumed to be part of the list. Lists also allow trailing commas, meaning you can have a comma with nothing following it at the end of a list. The interpreter knows to ignore it.

```
long_list = [1, 2, 3, 4, 5, 6, 7, 8, 9,
10, 11, 12, 13, 14, 15, 16, 17, 18, 19,
20, 21, 22, 23, 24, 25, 26, 27, 28, 29,
30, 31, 32, 33, 34, 35, 36, 37, 38, 39,]

months = [
    "January", "February", "March",
    "April", "May", "June",
    "July", "August", "September",
    "October", "November", "December"
]
```

Reading Elements

To access elements of the list, the subscript operator is used. The subscript operator is the square brackets. Like arrays in most languages, the indices for Python lists start at 0. The maximum index is one less than the length of the list. To access the length of the list, the `len()` function is provided.

```
print(lst[0]) # prints 0
print(lst[2]) # prints True

# the len() member function can be passed a container
# as an argument, and returns the number elements it holds
lst_size = len(lst)
print(lst[lst_size - 1])
```

Unlike other programming languages, Python supports a shorthand syntax for accessing elements at the back of the list: a negative index can be passed to the subscript operator. This index is added to the length of the list to determine which index is being accessed. For example, `lst[-1]` is equivalent to `lst[len(lst) + (-1)]`. In other

words, `lst[-1]` is used to access the last element of the list. `lst[-2]` is used to access the second to last element, and so on. Another feature offered by Python's subscript operator is slicing. Slicing allows a range of elements to be selected from a list. To slice elements, you separate the start index and end index of the range you want to slice with a colon. The element at the end index is not included in the slice:

```
# this creates a new list with the elements
# at lst[0], lst[1], and lst[2]
# the end index 3 is not included in the slice
sublist = lst[0:3]
assert sublist == [0, "hello", True]

# not putting a value before the colon
# is shorthand for the first element
sublist2 = lst[:3]
assert sublist2 == [0, "hello", True]

# since the end index is exclusive, passing the
# length of the list will allow you to copy the
# last element of the list
sublist3 = lst[3:lst_size]
assert sublist3 == ["banana", 4.7]

# a shorthand for copying to the end of the list is to
# not put a value after the colon
sublist4 = lst[3:]
assert sublist4 == ["banana", 4.7]
```

Modifying Elements

The subscript operator can be used to modify existing elements in a list:

```
lst[0] = "apple"
lst[-1] = "orange"
lst[2] = "grape"
lst[1] = "cherry"
assert lst == ["apple", "cherry", "grape", "banana", "orange"]
```

Inserting Elements

To add new elements to a list, the `append()` function is used. This inserts a new element at the end of the list. If you want to add an element at a specific index, the `insert()` function can be used.

`insert()` expects two arguments, the index to insert at, and the

element to be inserted.

```
# this creates an empty list
lst = []

# adds the numbers 0 to 9 to the list
for i in range(10):
    lst.append(i)

# insert 0.5 into the list after 0
# since 0 is at index 0, we insert at index 1
lst.insert(1, 0.5)
assert lst[0] == 0
assert lst[1] == 0.5
assert lst[2] == 1

# you can insert at the end of the list
# by passing len(lst) as the index, as well
# although, just using append is easier for this
lst.insert(len(lst), 10)
assert lst[-1] == 10

# negative indexes can also be passed
# they function the same as they do for the subscript
lst.insert(-1, 9.5)
assert lst[-3] == 9
assert lst[-2] == 9.5
assert lst[-1] == 10

# this works, but might not do what you think it will
lst.append([11, 12, 13, 14])

# lists can hold ANYTHING in Python, including other lists
# when appending a list to a list, the list itself is appended
assert lst == [
    0, 0.5, 1, 2, 3, 4, 5, 6, 7, 8, 9, 9.5, 10,
    [11, 12, 13, 14]
]
```

If you want to just add the values of a list to the end of another list, the `extend()` member function is used. Below shows what the result of using `extend` instead of `append` would be:

```
lst.extend([11, 12, 13, 14])

# now, 11, 12, 13, 14 are individual elements of lst,
# rather than being elements of a list contained in lst
assert lst == [
```

```
0, 0.5, 1, 2, 3, 4, 5, 6, 7, 8, 9, 9.5, 10,  
11, 12, 13, 14  
]
```

Removing Elements

To remove all elements from a list, the `clear()` function is used. To remove a particular element from a list, the `remove()` function is used. The value of the element that is to be removed is passed as an argument to `remove()`. An exception called `ValueError` will be thrown if you attempt to remove an element not in the list. We'll talk about exception handling in a few weeks. Only the first instance of an element is removed by `remove()`, even if there are multiple instances of it in the list. To remove an item at a specific index, the `pop()` member function is used. If no index is provided as an argument, `pop` removes the last element. If the index is invalid or the list is empty, a `IndexError` exception is thrown.

```
lst = ["apple", "orange", "banana", "grape", "apple", "mango"]  
  
# this will only remove the first "apple",  
# which happens to be the first element in the list  
lst.remove("apple")  
assert lst[0] == "orange"  
# the second instance of apple is still in the list  
assert lst[3] == "apple"  
assert len(lst) == 5  
  
# this removes the last element, "mango"  
lst.pop()  
assert lst[-1] == "apple"  
assert len(lst) == 4  
  
# this removes the first element, "orange"  
lst.pop(0)  
assert lst[0] == "banana"  
assert len(lst) == 3  
  
# this removes all remaining elements  
lst.clear()  
assert lst == []  
assert len(lst) == 0
```

Other List Functions

The `sort()` function is used to sort the elements of a list. The `reverse()` function reverses the contents of a list. Both of these occur "in-place", meaning they impact the list they are called on. The `index()` function is passed an element as an argument, and returns that element's index in the list. If the element is not in the list, a `ValueError` is raised. The `count()` function returns the number of times an element occurs in a list.

Copying Lists

We now need to talk a bit about how variables and memory are handled in Python. The "primitive" data types, int, float, str, and bool, are copied when you assign a variable to an existing variable:

```
x = 2
y = x
y = 4
assert x == 2
assert y == 4
```

In this case, changes to y do not impact x. More complex data types, like lists, however, are shallow copied. If the list only contains primitive types, this is not a problem:

```
lst = [0, "hello", 5.4]
other = lst
other[1] = ("world")

assert lst == [0, "hello", 5.4]
assert other == [0, "world", 5.4]
```

However, if one of the elements are a complex type, like a list, the shallow copy can cause discrepancies:

```
lst = [1, 2, [3, 4, 5]]
other = lst

other[2].append(6)

assert lst == [1, 2, [3, 4, 5, 6]]
assert other == [1, 2, [3, 4, 5, 6]]
```

Both `lst` and `other` are sharing the same copy of the list `[3, 4, 5]` in memory. Therefore, when `other` makes a change to that list, such as adding an element, it modifies `lst`. Similarly, if we made a change to that list using `lst`, `other` would be impacted. If we want to make an entirely new version of the list, also called a deep copy, we need to import the `copy` module and use the `copy.deepcopy()` function:

```
import copy

lst = [1, 2, [3, 4, 5]]
other = copy.deepcopy(lst)

other[2].append(6)

assert lst == [1, 2, [3, 4, 5]]
assert other == [1, 2, [3, 4, 5, 6]]
```

Deep copying large objects can be very expensive both computationally and in memory usage, so you need to determine whether deep copying is actually needed when you copy a list.

List Comprehensions

Earlier, there was an example of using a for loop to append the values from 0 to 9 into a list. Python provides a shorthand for doing this, called a list comprehension. Below is two equivalent ways of inserting the values from 1 to 100 into a list:

```
for_list = []
# using a traditional for loop
for i in range(1, 101):
    for_list.append(i)

# using a list comprehension
comp_list = [i for i in range(1, 101)]

assert for_list == comp_list
```

The list comprehension consists of a for loop within the square brackets for the list. The expression that appears before the for loop is what is appended to the list each time the loop is run. In the example above, it is just appending the value of `i`. However, it can be any expression.

```
# appends i^2 for i = 0 to 19
list_of_squares = [i ** 2 for i in range(0, 20)]
```

List comprehensions are considered the "idiomatic" or "Pythonic" way of generating lists. Idioms are commonly accepted ways of expressing something in a programming language. While the traditional for loop is syntactically correct and is no less efficient than a comprehension, Python developers still prefer comprehensions.

Using For Loops to Iterate over Lists

If we want to iterate over a Python list, we could use a normal counter controlled while or for loop to access each index of the list:

```
lst = [i for i in range(10)]

# printing elements in order
for i in range(len(lst)):
    print(lst[i])

# printing elements in reverse order
i = len(lst) - 1
while i >= 0:
    print(lst[i])
```

However, recall that Python for loops are range based. This means that they are designed to iterate over collections. That is why we use the `range()` function to return a collection of values to iterate over.

Rather than generating a range and passing each index, we can simply iterate over the list directly:

```
lst = [i for i in range(10)]

for x in lst:
    print(x)
```

Tuples

A tuple, like a list, is a sequential container. The difference is that it cannot be changed after it is created. No new elements can be inserted, nor can existing elements be removed. Rather than being declared with square brackets, Python tuples use parentheses:


```
tupl = (1, 2, 3, 4)
```

You can create a tuple from a list by using the `tuple()` constructor function:

```
lst = [i for i in range(10)]  
tupl = tuple(lst)
```

You can create a list from a tuple using the `list()` constructor function:

```
tupl = (1, 2, 3, 4, 5)  
lst = list(tupl)
```

Elements of tuples can be read using the subscript operator, just like lists. However, the subscript cannot directly modify the elements of a tuple. Slicing also works with tuples. The slice of a tuple is another tuple:

```
tupl = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)  
  
# this is fine, we are just reading the value  
print(tupl[5]) # prints 6  
  
# this is not, we are trying to overwrite a value  
# tuples do not allow values to be overwritten  
# tupl[5] = 4  
  
# slicing a tuple yields another tuple  
# if you want a list, you can use the list() function  
subtupl = tupl[1:5]  
assert subtupl == (2, 3, 4, 5)  
  
lst = list(subtupl)  
assert lst == [2, 3, 4, 5]
```

Like with lists, things get weird when the elements of a tuple are more complex objects, like another list. Since a list is a modifiable container, a list that is an element of a tuple can still be modified, even though the tuple is supposed to be immutable:

```
tupl = (1, 2, [3, 4, 5])
```

```
# this can be done, since it is modifying the list
tupl[2].append(6)
assert tupl[2] == [3, 4, 5, 6]

# this can't be done, since it is directly
# modifying the tuple's element
# tupl[2] = [3, 4, 5, 6, 7]
```

So, why is the first modification legal, but not the second? In the first instance, we are modifying the list itself. At this point, Python does not care that the list at `tupl[2]` is part of a tuple. It only sees that you are appending a value to a list. In the second version, we are saying to take `tupl[2]`, and replace the value at `tupl[2]` with a new element. It doesn't matter that we are trying to replace a list with another list. As far as Python cares, we are directly changing the element of a tuple, which is illegal, and will raise an exception.

Lists and Tuples as Function Parameters

Functions can accept lists and tuples as arguments. When lists and tuples are passed to a function, they are passed by reference. Since tuples are (mostly) immutable, this is not much of a problem. For lists, though, this means that changes to the list inside the function will be reflected outside the function:

```
def extend_list(lst):
    lst.extend([i for i in range(len(lst), len(lst) + 10)])

lst = [i for i in range(0, 10)]
print(lst)

# [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

extend_list(lst)
print(lst)
# [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10,
# 11, 12, 13, 14, 15, 16, 17, 18, 19]
```

Functions can also return lists or tuples:

```
def get_list(max):
    return [i for i in range(max + 1)]
```

```
lst = get_list(10)
assert lst == [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Conclusion

This week, we discussed lists and tuples in Python. Lists are similar to arrays in other programming languages in how elements are accessed, but lists do not have a fixed size, and can hold elements of any data type. We also discussed tuples, which are like lists, except they cannot be changed after they are created. Lists and tuples can be used as parameters to and return values of functions.